

MMALS_Prototype_0_11_Comparative_Evidence_Harness

May 11, 2026

1 MMALS Prototype v0.11 — Comparative Evidence Harness

Mycelium-Inspired Mutualistic Adaptive Learning Systems

1.1 Purpose

v0.8 showed that MMALS can learn sequential tasks on Fashion-MNIST without obvious catastrophic forgetting.

v0.9 diagnosed what memory means inside MMALS: route alone vs mediated latent function.

v0.10.1 measured compute/memory trade-offs of full functional memory.

v0.11 is different.

It is not a new MMALS improvement.

It is a comparative benchmark harness designed to answer:

Does MMALS provide a real continual-learning advantage compared with known baselines?

1.2 Baseline families included

1. Lower / upper reference baselines: fine-tuning, joint training upper bound.
2. Regularization methods: EWC, simplified SI scaffold, LwF.
3. Memory / replay methods: Experience Replay, iCaRL scaffold, simplified AGEM scaffold.
4. Expansion / modularity methods: Progressive Neural Network style baseline, Sparse Mixture-of-Experts baseline.
5. Standard deep models: MLP, Simple CNN, Tiny ResNet-style model, Tiny ViT-style model.
6. MMALS variants: `distill_only`, `latent_only`, `route_plus_latent`, `full_functional_memory`.
7. Optional graph-learning track: GCN baseline on Cora if `torch_geometric` is available.

1.3 Metrics

- Final average accuracy
- Average forgetting
- Per-task final accuracy
- Train time
- Inference latency scaffold
- Parameter count
- Peak GPU memory
- Replay memory size

- Relative efficiency score

```
[1]: # =====
# 0. Imports and Colab-safe setup
# =====

import math
import copy
import random
import gc
import time
from pathlib import Path
from typing import Dict, List, Tuple, Optional, Any

import numpy as np
import pandas as pd

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import TensorDataset, DataLoader

from torchvision import datasets, transforms

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("DEVICE:", DEVICE)
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))

def set_seed(seed: int):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

def clear_memory():
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()

def reset_peak_memory():
    if torch.cuda.is_available():
        torch.cuda.reset_peak_memory_stats()

def get_peak_memory_mb():
    if torch.cuda.is_available():
```

```

        return torch.cuda.max_memory_allocated() / (1024 ** 2)
    return np.nan

def sync_cuda():
    if torch.cuda.is_available():
        torch.cuda.synchronize()

def make_loader(ds, batch_size=128, shuffle=True):
    return DataLoader(ds, batch_size=batch_size, shuffle=shuffle,
        ↪ num_workers=0, pin_memory=False)

def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

def model_size_mb(model, bytes_per_param=4):
    return count_parameters(model) * bytes_per_param / (1024**2)

@torch.no_grad()
def sample_batch(dataset, batch_size=128):
    X, y = dataset.tensors
    n = len(dataset)
    k = min(batch_size, n)
    idx = torch.randint(0, n, (k,))
    return X[idx].to(DEVICE), y[idx].to(DEVICE)

```

DEVICE: cpu

2 1. Run modes

Use three modes:

- **smoke**: checks that all selected baselines execute.
- **evidence**: recommended first serious experiment.
- **paper**: heavier multi-seed experiment suitable for reporting.

The notebook includes all comparison families, but you should not run the full matrix at once until the pipeline is validated.

```

[2]: # =====
# 1. Configuration
# =====

RUN_MODE = "smoke" # "smoke", "evidence", "paper"
DATASET_NAME = "FashionMNIST" # "MNIST", "FashionMNIST", "CIFAR10"
DATA_ROOT = "./data"

if RUN_MODE == "smoke":
    TRAIN_PER_CLASS = 200

```

```

TEST_PER_CLASS = 100
SEEDS = [0]
EPOCHS_PER_TASK = 2
BATCH_SIZE = 128
SELECTED_METHODS = [
    "finetune",
    "joint_upper_bound",
    "ewc",
    "lwf",
    "experience_replay",
    "pnn",
    "sparse_moe",
    "mmals_full_functional_memory",
]
SELECTED_BACKBONES = ["mlp"]

elif RUN_MODE == "evidence":
    TRAIN_PER_CLASS = 1200
    TEST_PER_CLASS = 400
    SEEDS = [0, 1, 2]
    EPOCHS_PER_TASK = 8
    BATCH_SIZE = 128
    SELECTED_METHODS = [
        "finetune",
        "joint_upper_bound",
        "ewc",
        "si_simplified",
        "lwf",
        "experience_replay",
        "agem_simplified",
        "pnn",
        "sparse_moe",
        "mmals_distill_only",
        "mmals_latent_only",
        "mmals_route_plus_latent",
        "mmals_full_functional_memory",
    ]
    SELECTED_BACKBONES = ["mlp", "cnn"]

elif RUN_MODE == "paper":
    TRAIN_PER_CLASS = 3000
    TEST_PER_CLASS = 1000
    SEEDS = list(range(10))
    EPOCHS_PER_TASK = 15
    BATCH_SIZE = 128
    SELECTED_METHODS = [
        "finetune",

```

```

        "joint_upper_bound",
        "ewc",
        "si_simplified",
        "lwf",
        "experience_replay",
        "icarl_scaffold",
        "agem_simplified",
        "pnn",
        "sparse_moe",
        "mmals_distill_only",
        "mmals_latent_only",
        "mmals_route_plus_latent",
        "mmals_full_functional_memory",
    ]
    SELECTED_BACKBONES = ["mlp", "cnn", "tiny_resnet", "tiny_vit"]

else:
    raise ValueError("RUN_MODE must be smoke, evidence, or paper")

N_TASKS = 5
LR = 1e-3
INPUT_DIM = 28 * 28 if DATASET_NAME in ["MNIST", "FashionMNIST"] else 3 * 32 * 32
N_CHANNELS = 1 if DATASET_NAME in ["MNIST", "FashionMNIST"] else 3
IMAGE_SIZE = 28 if DATASET_NAME in ["MNIST", "FashionMNIST"] else 32

print("RUN_MODE:", RUN_MODE)
print("DATASET_NAME:", DATASET_NAME)
print("TRAIN_PER_CLASS:", TRAIN_PER_CLASS)
print("TEST_PER_CLASS:", TEST_PER_CLASS)
print("SEEDS:", SEEDS)
print("EPOCHS_PER_TASK:", EPOCHS_PER_TASK)
print("SELECTED_METHODS:", SELECTED_METHODS)
print("SELECTED_BACKBONES:", SELECTED_BACKBONES)

```

```

RUN_MODE: smoke
DATASET_NAME: FashionMNIST
TRAIN_PER_CLASS: 200
TEST_PER_CLASS: 100
SEEDS: [0]
EPOCHS_PER_TASK: 2
SELECTED_METHODS: ['finetune', 'joint_upper_bound', 'ewc', 'lwf',
'experience_replay', 'pnn', 'sparse_moe', 'mmals_full_functional_memory']
SELECTED_BACKBONES: ['mlp']

```

3 2. Dataset builders

The primary v0.11 image benchmark uses sequential binary tasks:

- Task 0: class 0 vs 1
- Task 1: class 2 vs 3
- Task 2: class 4 vs 5
- Task 3: class 6 vs 7
- Task 4: class 8 vs 9

This supports task-incremental comparison. Later versions should add class-incremental evaluation without task id.

```
[3]: # =====  
# 2. Dataset builders  
# =====  
  
def get_transform(name, flatten=True):  
    base = [transforms.ToTensor()]  
    if flatten:  
        base.append(transforms.Lambda(lambda x: x.view(-1)))  
    return transforms.Compose(base)  
  
def load_base_dataset(name, flatten=True):  
    transform = get_transform(name, flatten=flatten)  
    if name == "MNIST":  
        train = datasets.MNIST(DATA_ROOT, train=True, download=True,  
→transform=transform)  
        test = datasets.MNIST(DATA_ROOT, train=False, download=True,  
→transform=transform)  
        elif name == "FashionMNIST":  
            train = datasets.FashionMNIST(DATA_ROOT, train=True, download=True,  
→transform=transform)  
            test = datasets.FashionMNIST(DATA_ROOT, train=False, download=True,  
→transform=transform)  
            elif name == "CIFAR10":  
                train = datasets.CIFAR10(DATA_ROOT, train=True, download=True,  
→transform=transform)  
                test = datasets.CIFAR10(DATA_ROOT, train=False, download=True,  
→transform=transform)  
            else:  
                raise ValueError("DATASET_NAME must be MNIST, FashionMNIST, or CIFAR10")  
            return train, test  
  
def get_labels(base_dataset):  
    if hasattr(base_dataset, "targets"):  
        return np.array(base_dataset.targets)  
    raise ValueError("Dataset has no targets attribute.")
```

```

def subset_pair_dataset(base_dataset, a, b, max_per_class=None, seed=42):
    rng = np.random.default_rng(seed)
    labels = get_labels(base_dataset)

    idx_a = np.where(labels == a)[0]
    idx_b = np.where(labels == b)[0]

    if max_per_class is not None:
        idx_a = rng.choice(idx_a, size=min(max_per_class, len(idx_a)),
↪replace=False)
        idx_b = rng.choice(idx_b, size=min(max_per_class, len(idx_b)),
↪replace=False)

    indices = np.concatenate([idx_a, idx_b])
    rng.shuffle(indices)

    Xs, ys = [], []
    for idx in indices:
        x, y = base_dataset[int(idx)]
        Xs.append(x)
        ys.append(1 if int(y) == b else 0)

    return TensorDataset(torch.stack(Xs).float(), torch.tensor(ys).long())

def build_split_image_tasks(name=DATASET_NAME, seed=42, n_tasks=N_TASKS,
↪flatten=True):
    train_base, test_base = load_base_dataset(name, flatten=flatten)
    pairs = [(0, 1), (2, 3), (4, 5), (6, 7), (8, 9)][:n_tasks]
    tasks = []

    for task_id, (a, b) in enumerate(pairs):
        train_ds = subset_pair_dataset(
            train_base, a, b, max_per_class=TRAIN_PER_CLASS, seed=seed + task_id
        )
        test_ds = subset_pair_dataset(
            test_base, a, b, max_per_class=TEST_PER_CLASS, seed=seed + 100 +
↪task_id
        )
        tasks.append({
            "task_id": task_id,
            "name": f"{a} vs {b}",
            "pair": (a, b),
            "train": train_ds,
            "test": test_ds,
        })

```

```

    return tasks

tasks_preview = build_split_image_tasks(DATASET_NAME, seed=0, flatten=True)
[(t["task_id"], t["name"], len(t["train"]), len(t["test"])) for t in
 ↪tasks_preview]

```

```

100%|      | 26.4M/26.4M [00:01<00:00, 21.7MB/s]
100%|      | 29.5k/29.5k [00:00<00:00, 345kB/s]
100%|      | 4.42M/4.42M [00:00<00:00, 6.28MB/s]
100%|      | 5.15k/5.15k [00:00<00:00, 15.4MB/s]

```

```

[3]: [(0, '0 vs 1', 400, 200),
      (1, '2 vs 3', 400, 200),
      (2, '4 vs 5', 400, 200),
      (3, '6 vs 7', 400, 200),
      (4, '8 vs 9', 400, 200)]

```

4 3. Standard backbones

v0.11 separates the **learning algorithm** from the **backbone**:

- MLP: cheap and comparable to previous MMALS runs.
- CNN: stronger vision baseline.
- Tiny ResNet: checks if residual inductive bias explains gains.
- Tiny ViT: checks whether attention-style routing dominates the mycelial router.

In smoke mode, start with MLP only.

```

[4]: # =====
# 3. Backbones and task-head model
# =====

class MLPBackbone(nn.Module):
    def __init__(self, input_dim=INPUT_DIM, hidden_dim=256, latent_dim=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, latent_dim),
            nn.ReLU(),
        )
        self.output_dim = latent_dim

    def forward(self, x):
        if x.ndim > 2:
            x = x.view(x.shape[0], -1)
        return self.net(x)

```



```

class SimpleCNNBackbone(nn.Module):
    def __init__(self, n_channels=N_CHANNELS, image_size=IMAGE_SIZE,
        ↪latent_dim=64):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(n_channels, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
        dummy = torch.zeros(1, n_channels, image_size, image_size)
        with torch.no_grad():
            flat_dim = self.conv(dummy).view(1, -1).shape[1]
        self.fc = nn.Sequential(
            nn.Linear(flat_dim, 128),
            nn.ReLU(),
            nn.Linear(128, latent_dim),
            nn.ReLU(),
        )
        self.output_dim = latent_dim

    def forward(self, x):
        if x.ndim == 2:
            x = x.view(x.shape[0], N_CHANNELS, IMAGE_SIZE, IMAGE_SIZE)
        return self.fc(self.conv(x).view(x.shape[0], -1))

class TinyResidualBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(channels, channels, 3, padding=1),
            nn.ReLU(),
            nn.Conv2d(channels, channels, 3, padding=1),
        )

    def forward(self, x):
        return F.relu(x + self.block(x))

class TinyResNetBackbone(nn.Module):
    def __init__(self, n_channels=N_CHANNELS, image_size=IMAGE_SIZE,
        ↪latent_dim=64):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(n_channels, 32, 3, padding=1),
            nn.ReLU(),

```

```

        TinyResidualBlock(32),
        nn.MaxPool2d(2),
        nn.Conv2d(32, 64, 3, padding=1),
        nn.ReLU(),
        TinyResidualBlock(64),
        nn.MaxPool2d(2),
    )
    dummy = torch.zeros(1, n_channels, image_size, image_size)
    with torch.no_grad():
        flat_dim = self.stem(dummy).view(1, -1).shape[1]
    self.fc = nn.Sequential(
        nn.Linear(flat_dim, 128),
        nn.ReLU(),
        nn.Linear(128, latent_dim),
        nn.ReLU(),
    )
    self.output_dim = latent_dim

    def forward(self, x):
        if x.ndim == 2:
            x = x.view(x.shape[0], N_CHANNELS, IMAGE_SIZE, IMAGE_SIZE)
        return self.fc(self.stem(x).view(x.shape[0], -1))

class TinyViTBackbone(nn.Module):
    def __init__(self, n_channels=N_CHANNELS, image_size=IMAGE_SIZE,
        ↪ patch_size=7, d_model=64, nhead=4, layers=2):
        super().__init__()
        assert image_size % patch_size == 0
        self.n_channels = n_channels
        self.image_size = image_size
        self.patch_size = patch_size
        self.num_patches = (image_size // patch_size) ** 2
        self.patch_dim = n_channels * patch_size * patch_size

        self.proj = nn.Linear(self.patch_dim, d_model)
        self.pos = nn.Parameter(torch.zeros(1, self.num_patches, d_model))
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=nhead, dim_feedforward=128, batch_first=True
        )
        self.encoder = nn.TransformerEncoder(encoder_layer, num_layers=layers)
        self.output_dim = d_model

    def forward(self, x):
        if x.ndim == 2:
            x = x.view(x.shape[0], self.n_channels, self.image_size, self.
            ↪ image_size)

```

```

        patches = x.unfold(2, self.patch_size, self.patch_size).unfold(3, self.
↪patch_size, self.patch_size)
        patches = patches.contiguous().view(x.shape[0], self.n_channels, -1,
↪self.patch_size, self.patch_size)
        patches = patches.permute(0, 2, 1, 3, 4).contiguous().view(x.shape[0],
↪self.num_patches, -1)

        z = self.proj(patches) + self.pos
        z = self.encoder(z)
        return z.mean(dim=1)

class TaskHeadModel(nn.Module):
    def __init__(self, backbone_name="mlp", n_tasks=N_TASKS, latent_dim=64):
        super().__init__()
        self.backbone_name = backbone_name

        if backbone_name == "mlp":
            self.backbone = MLPBackbone(latent_dim=latent_dim)
        elif backbone_name == "cnn":
            self.backbone = SimpleCNNBackbone(latent_dim=latent_dim)
        elif backbone_name == "tiny_resnet":
            self.backbone = TinyResNetBackbone(latent_dim=latent_dim)
        elif backbone_name == "tiny_vit":
            self.backbone = TinyViTBackbone(d_model=latent_dim)
        else:
            raise ValueError(f"Unknown backbone: {backbone_name}")

        self.heads = nn.ModuleList([nn.Linear(self.backbone.output_dim, 2) for
↪_ in range(n_tasks)])

    def forward(self, x, task_id):
        z = self.backbone(x)
        return self.heads[task_id](z)

    def features(self, x):
        return self.backbone(x)

```

5 4. MMALS model carried from v0.9 / v0.10.1

This preserves the current MMALS core:

$$z_f(x,t) = \sum_i r_i(x,t) T_i(g_i(x))$$

v0.11 uses it as one competitor among others, not as the whole experiment.

```

[5]: # =====
     # 4. MMALS core model

```

```
# =====

class MutualisticMMALS11(nn.Module):
    def __init__(
        self,
        input_dim=INPUT_DIM,
        hidden_dim=256,
        latent_dim=64,
        route_hidden_dim=128,
        n_tasks=N_TASKS,
        n_hosts=5,
        task_emb_dim=16,
        temperature=0.65,
    ):
        super().__init__()
        self.n_tasks = n_tasks
        self.n_hosts = n_hosts
        self.temperature = temperature

        self.hosts = nn.ModuleList([
            nn.Sequential(
                nn.Linear(input_dim, hidden_dim),
                nn.ReLU(),
                nn.Linear(hidden_dim, latent_dim),
                nn.ReLU(),
            )
            for _ in range(n_hosts)
        ])

        self.transforms = nn.ModuleList([
            nn.Sequential(
                nn.Linear(latent_dim, latent_dim),
                nn.ReLU(),
                nn.Linear(latent_dim, latent_dim),
            )
            for _ in range(n_hosts)
        ])

        self.task_embedding = nn.Embedding(n_tasks, task_emb_dim)
        self.router = nn.Sequential(
            nn.Linear(input_dim + task_emb_dim, route_hidden_dim),
            nn.ReLU(),
            nn.Linear(route_hidden_dim, n_hosts),
        )

        self.heads = nn.ModuleList([nn.Linear(latent_dim, 2) for _ in
↪range(n_tasks)])
```

```

def route(self, x, task_id):
    if x.ndim > 2:
        x = x.view(x.shape[0], -1)
        task_ids = torch.full((x.shape[0],), task_id, dtype=torch.long,
device=x.device)
        e = self.task_embedding(task_ids)
        route_input = torch.cat([x, e], dim=1)
        logits = self.router(route_input)
        return F.softmax(logits / self.temperature, dim=1)

def host_outputs(self, x):
    if x.ndim > 2:
        x = x.view(x.shape[0], -1)
    return torch.stack([host(x) for host in self.hosts], dim=1)

def transformed_outputs(self, host_z):
    transformed = []
    for i, transform in enumerate(self.transforms):
        transformed.append(transform(host_z[:, i, :]))
    return torch.stack(transformed, dim=1)

def mixture_from(self, transformed, r):
    return torch.sum(r.unsqueeze(-1) * transformed, dim=1)

def forward_with_details(self, x, task_id):
    r = self.route(x, task_id)
    host_z = self.host_outputs(x)
    transformed = self.transformed_outputs(host_z)
    z_mix = self.mixture_from(transformed, r)
    logits = self.heads[task_id](z_mix)
    return logits, r, transformed, z_mix

def forward(self, x, task_id):
    logits, _, _, _ = self.forward_with_details(x, task_id)
    return logits

def route_entropy(r, eps=1e-8):
    return -(r * torch.log(r + eps)).sum(dim=1).mean()

def route_dominance(r):
    return r.max(dim=1).values.mean()

def metabolic_cost(r, transformed):
    norms = (transformed ** 2).sum(dim=2)
    return (r * norms).sum(dim=1).mean()

def kl_distill_loss(student_logits, teacher_logits, temperature=2.0):

```

```

T = temperature
student_logp = F.log_softmax(student_logits / T, dim=1)
teacher_p = F.softmax(teacher_logits / T, dim=1)
return F.kl_div(student_logp, teacher_p, reduction="batchmean") * (T * T)

@torch.no_grad()
def compute_average_route(model, dataset, task_id, batch_size=256):
    model.eval()
    loader = make_loader(dataset, batch_size=batch_size, shuffle=False)
    routes = []
    for xb, _ in loader:
        xb = xb.to(DEVICE)
        r = model.route(xb, task_id)
        routes.append(r.detach().cpu())
    return torch.cat(routes, dim=0).mean(dim=0)

def mmals_loss(
    model,
    x,
    y,
    task_id,
    old_task_ids,
    tasks,
    route_targets=None,
    teacher=None,
    lambda_entropy=0.006,
    lambda_metabolic=0.0005,
    lambda_route_memory=2.0,
    lambda_distill=1.0,
    lambda_latent=1.0,
    distill_temperature=2.0,
):
    logits, r, transformed, z_mix = model.forward_with_details(x, task_id)
    total = F.cross_entropy(logits, y)
    total = total + lambda_entropy * route_entropy(r)
    total = total + lambda_metabolic * metabolic_cost(r, transformed)

    if lambda_route_memory > 0 and route_targets is not None and old_task_ids:
        route_losses = []
        for old_tid in old_task_ids:
            if old_tid not in route_targets:
                continue
            xb_old, _ = sample_batch(tasks[old_tid]["train"],
↪batch_size=BATCH_SIZE)
            r_current = model.route(xb_old, old_tid).mean(dim=0)
            r_target = route_targets[old_tid].to(DEVICE)
            route_losses.append(F.mse_loss(r_current, r_target))

```

```

        if route_losses:
            total = total + lambda_route_memory * torch.stack(route_losses).
↪mean()

        if teacher is not None and old_task_ids and (lambda_distill > 0 or
↪lambda_latent > 0):
            teacher.eval()
            distill_losses = []
            latent_losses = []
            for old_tid in old_task_ids:
                xb_old, _ = sample_batch(tasks[old_tid]["train"],
↪batch_size=BATCH_SIZE)
                student_logits, _, _, student_z = model.
↪forward_with_details(xb_old, old_tid)
                with torch.no_grad():
                    teacher_logits, _, _, teacher_z = teacher.
↪forward_with_details(xb_old, old_tid)
                    if lambda_distill > 0:
                        distill_losses.append(kl_distill_loss(student_logits,
↪teacher_logits, temperature=distill_temperature))
                    if lambda_latent > 0:
                        latent_losses.append(F.mse_loss(student_z, teacher_z))
            if distill_losses:
                total = total + lambda_distill * torch.stack(distill_losses).mean()
            if latent_losses:
                total = total + lambda_latent * torch.stack(latent_losses).mean()

    return total

```

6 5. Baseline mechanisms

These are research-harness implementations, not optimized library implementations.

For final paper-grade results, cross-check at least EWC, SI, LwF, Replay and AGEM against a reference implementation.

```

[6]: # =====
# 5. Baseline mechanisms
# =====

@torch.no_grad()
def evaluate_model(model, tasks, upto_task=None):
    model.eval()
    if upto_task is None:
        upto_task = len(tasks) - 1

    rows = []

```

```

    for task_id in range(upto_task + 1):
        loader = make_loader(tasks[task_id]["test"], batch_size=BATCH_SIZE,
↪shuffle=False)
        correct, total = 0, 0
        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            logits = model(xb, task_id)
            pred = logits.argmax(dim=1)
            correct += (pred == yb).sum().item()
            total += yb.numel()

        rows.append({
            "task_id": task_id,
            "task_name": tasks[task_id]["name"],
            "acc": correct / max(1, total),
        })
    return rows

def compute_final_metrics(history_df, group_cols):
    tagged = history_df.copy()
    final_after = tagged.groupby(group_cols)["after_task"].max().
↪reset_index(name="final_after")
    tagged = tagged.merge(final_after, on=group_cols, how="left")
    final = tagged[tagged["after_task"] == tagged["final_after"]].copy()

    per_seed_acc = final.groupby(group_cols)["acc"].mean().
↪reset_index(name="final_mean_acc")

    forgetting_rows = []
    for keys, sub in tagged.groupby(group_cols + ["task_id"]):
        if not isinstance(keys, tuple):
            keys = (keys,)
        final_after_task = sub["final_after"].iloc[0]
        final_rows = sub[sub["after_task"] == final_after_task]
        if len(final_rows) == 0:
            continue
        best_acc = float(sub["acc"].max())
        final_acc = float(final_rows["acc"].iloc[0])
        row = dict(zip(group_cols + ["task_id"], keys))
        row["task_name"] = final_rows["task_name"].iloc[0]
        row["best_acc"] = best_acc
        row["final_acc"] = final_acc
        row["per_task_forgetting"] = best_acc - final_acc
        forgetting_rows.append(row)

    forgetting_df = pd.DataFrame(forgetting_rows)
    per_seed_forgetting = (

```



```

        forgetting_df.groupby(group_cols)["per_task_forgetting"]
        .mean()
        .reset_index(name="avg_forgetting")
    )

    return final, per_seed_acc, forgetting_df, per_seed_forgetting

class ReplayBuffer:
    def __init__(self, max_size=1000):
        self.max_size = max_size
        self.X = []
        self.y = []
        self.t = []

    def __len__(self):
        return len(self.y)

    def add_dataset(self, dataset, task_id, max_items=None):
        X, y = dataset.tensors
        n = len(y)
        if max_items is None:
            max_items = n
        idx = torch.randperm(n)[:max_items]
        for i in idx:
            self.X.append(X[i].cpu())
            self.y.append(y[i].cpu())
            self.t.append(task_id)

        if len(self.y) > self.max_size:
            keep = torch.randperm(len(self.y))[:self.max_size].tolist()
            self.X = [self.X[i] for i in keep]
            self.y = [self.y[i] for i in keep]
            self.t = [self.t[i] for i in keep]

    def sample(self, batch_size=128):
        if len(self.y) == 0:
            return None
        k = min(batch_size, len(self.y))
        idx = torch.randint(0, len(self.y), (k,))
        X = torch.stack([self.X[i] for i in idx]).to(DEVICE)
        y = torch.stack([self.y[i] for i in idx]).long().to(DEVICE)
        t = torch.tensor([self.t[i] for i in idx], dtype=torch.long,
        ↪device=DEVICE)
        return X, y, t

class EWCHelper:
    def __init__(self, model, lambda_ewc=500.0):

```

```

        self.lambda_ewc = lambda_ewc
        self.means = {}
        self.fisher = {}

    def consolidate(self, model, dataset, task_id, samples=512):
        model.eval()
        loader = make_loader(dataset, batch_size=BATCH_SIZE, shuffle=True)
        fisher = {n: torch.zeros_like(p, device=DEVICE) for n, p in model.
        ↪named_parameters() if p.requires_grad}
        count = 0

        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            model.zero_grad()
            loss = F.cross_entropy(model(xb, task_id), yb)
            loss.backward()
            bs = xb.shape[0]
            count += bs
            for n, p in model.named_parameters():
                if p.requires_grad and p.grad is not None:
                    fisher[n] += (p.grad.detach() ** 2) * bs
            if count >= samples:
                break

        for n in fisher:
            fisher[n] = fisher[n] / max(1, count)

        self.means = {n: p.detach().clone() for n, p in model.
        ↪named_parameters() if p.requires_grad}
        self.fisher = fisher

    def penalty(self, model):
        if not self.means:
            return torch.tensor(0.0, device=DEVICE)
        loss = torch.tensor(0.0, device=DEVICE)
        for n, p in model.named_parameters():
            if n in self.means:
                loss = loss + (self.fisher[n] * (p - self.means[n]) ** 2).sum()
        return self.lambda_ewc * loss

class SIHelper:
    def __init__(self, model, lambda_si=1.0, eps=1e-3):
        self.lambda_si = lambda_si
        self.eps = eps
        self.omega = {n: torch.zeros_like(p, device=DEVICE) for n, p in model.
        ↪named_parameters() if p.requires_grad}

```

```

        self.prev_params = {n: p.detach().clone() for n, p in model.
↪named_parameters() if p.requires_grad}

    def consolidate(self, model):
        for n, p in model.named_parameters():
            if p.requires_grad:
                delta = p.detach() - self.prev_params[n]
                self.omega[n] += delta.abs() / (delta.pow(2) + self.eps)
                self.prev_params[n] = p.detach().clone()

    def penalty(self, model):
        loss = torch.tensor(0.0, device=DEVICE)
        for n, p in model.named_parameters():
            if n in self.omega:
                loss = loss + (self.omega[n] * (p - self.prev_params[n]) ** 2).
↪sum()
        return self.lambda_si * loss

```

7 6. Sparse MoE and Progressive Network baselines

These baselines test whether MMALS is only benefiting from modularity, more capacity, routing, or expansion.

```

[7]: # =====
# 6. Sparse MoE and PNN-style baselines
# =====

class SparseMoEModel(nn.Module):
    def __init__(self, input_dim=INPUT_DIM, n_tasks=N_TASKS, n_experts=5,
↪hidden_dim=256, latent_dim=64, task_emb_dim=16):
        super().__init__()
        self.n_tasks = n_tasks
        self.n_experts = n_experts
        self.experts = nn.ModuleList([
            nn.Sequential(
                nn.Linear(input_dim, hidden_dim),
                nn.ReLU(),
                nn.Linear(hidden_dim, latent_dim),
                nn.ReLU(),
            )
            for _ in range(n_experts)
        ])
        self.task_embedding = nn.Embedding(n_tasks, task_emb_dim)
        self.router = nn.Sequential(
            nn.Linear(input_dim + task_emb_dim, 128),
            nn.ReLU(),

```

```

        nn.Linear(128, n_experts),
    )
    self.heads = nn.ModuleList([nn.Linear(latent_dim, 2) for _ in
↪range(n_tasks)])

    def route(self, x, task_id):
        if x.ndim > 2:
            x = x.view(x.shape[0], -1)
            tid = torch.full((x.shape[0],), task_id, dtype=torch.long, device=x.
↪device)
            emb = self.task_embedding(tid)
            return F.softmax(self.router(torch.cat([x, emb], dim=1)), dim=1)

    def forward(self, x, task_id):
        if x.ndim > 2:
            x = x.view(x.shape[0], -1)
            r = self.route(x, task_id)
            z = torch.stack([expert(x) for expert in self.experts], dim=1)
            z_mix = (r.unsqueeze(-1) * z).sum(dim=1)
            return self.heads[task_id](z_mix)

class PNNStyleModel(nn.Module):
    def __init__(self, input_dim=INPUT_DIM, n_tasks=N_TASKS, hidden_dim=256,
↪latent_dim=64):
        super().__init__()
        self.n_tasks = n_tasks
        self.columns = nn.ModuleList([
            nn.Sequential(
                nn.Linear(input_dim, hidden_dim),
                nn.ReLU(),
                nn.Linear(hidden_dim, latent_dim),
                nn.ReLU(),
                nn.Linear(latent_dim, 2),
            )
            for _ in range(n_tasks)
        ])

    def freeze_previous(self, current_task):
        for tid, col in enumerate(self.columns):
            requires_grad = (tid == current_task)
            for p in col.parameters():
                p.requires_grad = requires_grad

    def forward(self, x, task_id):
        if x.ndim > 2:
            x = x.view(x.shape[0], -1)
            return self.columns[task_id](x)

```

8 7. Training functions

Each method is trained in a common sequential loop.

The goal is not to perfectly optimize every baseline in v0.11 smoke mode.

The goal is to make the comparison matrix executable and auditable.

```
[8]: # =====
# 7. Training functions
# =====

def train_standard_method(method, backbone_name, seed, tasks):
    set_seed(seed)
    clear_memory()
    reset_peak_memory()

    if method == "sparse_moe":
        model = SparseMoEModel(n_tasks=len(tasks), n_experts=len(tasks)).
        to(DEVICE)
    elif method == "pnn":
        model = PNNStyleModel(n_tasks=len(tasks)).to(DEVICE)
    else:
        model = TaskHeadModel(backbone_name=backbone_name, n_tasks=len(tasks)).
        to(DEVICE)

    ewc = EWCHelper(model) if method == "ewc" else None
    si = SIHelper(model) if method == "si_simplified" else None
    replay = ReplayBuffer(max_size=1000) if method in ["experience_replay",
    "agem_simplified", "icarl_scaffold"] else None
    teacher = None

    history = []
    start = time.perf_counter()

    for task_id, task in enumerate(tasks):
        if method == "pnn":
            model.freeze_previous(task_id)

        opt = torch.optim.Adam([p for p in model.parameters() if p.
        requires_grad], lr=LR)
        loader = make_loader(task["train"], batch_size=BATCH_SIZE, shuffle=True)

        for epoch in range(EPOCHS_PER_TASK):
            model.train()
            for xb, yb in loader:
                xb, yb = xb.to(DEVICE), yb.to(DEVICE)
                opt.zero_grad()
```

```

logits = model(xb, task_id)
loss = F.cross_entropy(logits, yb)

if ewc is not None:
    loss = loss + ewc.penalty(model)

if si is not None:
    loss = loss + si.penalty(model)

if method == "lwf" and teacher is not None and task_id > 0:
    for old_tid in range(task_id):
        xb_old, _ = sample_batch(tasks[old_tid]["train"],
↪batch_size=BATCH_SIZE)
        with torch.no_grad():
            teacher_logits = teacher(xb_old, old_tid)
            student_logits = model(xb_old, old_tid)
            loss = loss + kl_distill_loss(student_logits,
↪teacher_logits, temperature=2.0)

if replay is not None and len(replay) > 0:
    replay_batch = replay.sample(batch_size=BATCH_SIZE)
    if replay_batch is not None:
        xr, yr, tr = replay_batch
        replay_losses = []
        for old_tid in tr.unique().tolist():
            mask = tr == old_tid
            replay_losses.append(F.
↪cross_entropy(model(xr[mask], int(old_tid)), yr[mask]))
        if replay_losses:
            replay_loss = torch.stack(replay_losses).mean()
            if method == "agem_simplified":
                loss = loss + 0.5 * replay_loss
            else:
                loss = loss + replay_loss

    loss.backward()
    opt.step()

if ewc is not None:
    ewc consolidate(model, task["train"], task_id)

if si is not None:
    si consolidate(model)

if replay is not None:
    replay.add_dataset(task["train"], task_id, max_items=200)

```

```

    if method == "lwf":
        teacher = copy.deepcopy(model).to(DEVICE).eval()
        for p in teacher.parameters():
            p.requires_grad = False

    acc_rows = evaluate_model(model, tasks, upto_task=task_id)
    for row in acc_rows:
        history.append({
            "seed": seed,
            "method": method,
            "backbone": backbone_name,
            "after_task": task_id,
            **row,
        })

    sync_cuda()
    train_time = time.perf_counter() - start
    peak_mem = get_peak_memory_mb()
    params = count_parameters(model)
    param_mb = model_size_mb(model)

    run_record = {
        "seed": seed,
        "method": method,
        "backbone": backbone_name,
        "param_count": params,
        "param_mb_fp32": param_mb,
        "train_time_s": train_time,
        "peak_gpu_mb": peak_mem,
        "replay_buffer_items": len(replay) if replay is not None else 0,
    }

    del model, teacher
    clear_memory()
    return pd.DataFrame(history), run_record

def train_joint_upper_bound(backbone_name, seed, tasks):
    set_seed(seed)
    clear_memory()
    reset_peak_memory()

    model = TaskHeadModel(backbone_name=backbone_name, n_tasks=len(tasks)).
    ↪to(DEVICE)
    opt = torch.optim.Adam(model.parameters(), lr=LR)

    all_loaders = [make_loader(t["train"], batch_size=BATCH_SIZE, shuffle=True)
    ↪for t in tasks]

```

```

start = time.perf_counter()

for epoch in range(EPOCHS_PER_TASK * len(tasks)):
    for task_id, loader in enumerate(all_loaders):
        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            opt.zero_grad()
            loss = F.cross_entropy(model(xb, task_id), yb)
            loss.backward()
            opt.step()

history = []
for after_task in range(len(tasks)):
    acc_rows = evaluate_model(model, tasks, upto_task=after_task)
    for row in acc_rows:
        history.append({
            "seed": seed,
            "method": "joint_upper_bound",
            "backbone": backbone_name,
            "after_task": after_task,
            **row,
        })

sync_cuda()
run_record = {
    "seed": seed,
    "method": "joint_upper_bound",
    "backbone": backbone_name,
    "param_count": count_parameters(model),
    "param_mb_fp32": model_size_mb(model),
    "train_time_s": time.perf_counter() - start,
    "peak_gpu_mb": get_peak_memory_mb(),
    "replay_buffer_items": 0,
}

del model
clear_memory()
return pd.DataFrame(history), run_record

```

9 8. MMALS training wrapper

This wrapper reuses the v0.9 / v0.10.1 logic and makes MMALS comparable in the same scorecard.

```

[9]: # =====
# 8. MMALS training wrapper
# =====

```



```

MMALS_CONFIGS = {
    "mmals_distill_only": {
        "lambda_route_memory": 0.0,
        "lambda_distill": 1.0,
        "lambda_latent": 0.0,
        "temperature": 0.70,
    },
    "mmals_latent_only": {
        "lambda_route_memory": 0.0,
        "lambda_distill": 0.0,
        "lambda_latent": 1.0,
        "temperature": 0.70,
    },
    "mmals_route_plus_latent": {
        "lambda_route_memory": 2.0,
        "lambda_distill": 0.0,
        "lambda_latent": 1.0,
        "temperature": 0.70,
    },
    "mmals_full_functional_memory": {
        "lambda_route_memory": 2.0,
        "lambda_distill": 1.0,
        "lambda_latent": 1.0,
        "temperature": 0.65,
    },
}

def train_mmals_method(method, seed, tasks):
    cfg = MMALS_CONFIGS[method]
    set_seed(seed)
    clear_memory()
    reset_peak_memory()

    model = MutualisticMMALS11(
        n_tasks=len(tasks),
        n_hosts=len(tasks),
        temperature=cfg["temperature"],
    ).to(DEVICE)

    opt = torch.optim.Adam(model.parameters(), lr=LR)
    route_targets = {} if cfg["lambda_route_memory"] > 0 else None
    teacher = None
    old_task_ids = []

    history = []
    start = time.perf_counter()

```

```

for task_id, task in enumerate(tasks):
    loader = make_loader(task["train"], batch_size=BATCH_SIZE, shuffle=True)

    for epoch in range(EPOCHS_PER_TASK):
        model.train()
        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            opt.zero_grad()
            loss = mmals_loss(
                model=model,
                x=xb,
                y=yb,
                task_id=task_id,
                old_task_ids=old_task_ids,
                tasks=tasks,
                route_targets=route_targets,
                teacher=teacher,
                lambda_route_memory=cfg["lambda_route_memory"],
                lambda_distill=cfg["lambda_distill"],
                lambda_latent=cfg["lambda_latent"],
            )
            loss.backward()
            opt.step()

        if route_targets is not None:
            route_targets[task_id] = compute_average_route(model,
↳task["train"], task_id)

        if cfg["lambda_distill"] > 0 or cfg["lambda_latent"] > 0:
            teacher = copy.deepcopy(model).to(DEVICE).eval()
            for p in teacher.parameters():
                p.requires_grad = False

        old_task_ids = list(range(task_id + 1))

        acc_rows = evaluate_model(model, tasks, upto_task=task_id)
        for row in acc_rows:
            history.append({
                "seed": seed,
                "method": method,
                "backbone": "mmals",
                "after_task": task_id,
                **row,
            })

sync_cuda()

```

```

run_record = {
    "seed": seed,
    "method": method,
    "backbone": "mmals",
    "param_count": count_parameters(model),
    "param_mb_fp32": model_size_mb(model),
    "train_time_s": time.perf_counter() - start,
    "peak_gpu_mb": get_peak_memory_mb(),
    "replay_buffer_items": 0,
}

del model, teacher
clear_memory()
return pd.DataFrame(history), run_record

```

10 9. Execute comparative experiment

Recommended use:

1. Run smoke first.
2. Check that all methods produce rows.
3. Then run evidence.
4. Only then run paper.

```

[10]: # =====
# 9. Run experiment
# =====

all_histories = []
run_records = []

run_idx = 0

for seed in SEEDS:
    tasks_seed = build_split_image_tasks(DATASET_NAME, seed=seed, flatten=True)

    for backbone_name in SELECTED_BACKBONES:
        for method in SELECTED_METHODS:
            if method.startswith("mmals") and backbone_name != SELECTED_BACKBONES[0]:
                continue
            if method in ["sparse_moe", "pnn"] and backbone_name != SELECTED_BACKBONES[0]:
                continue

            run_idx += 1

```

```

        print(f"\nRun {run_idx}: seed={seed}, backbone={backbone_name},  

        ↪method={method}")

        if method == "joint_upper_bound":
            hist, record = train_joint_upper_bound(backbone_name, seed,  

            ↪tasks_seed)
        elif method.startswith("mmals"):
            hist, record = train_mmals_method(method, seed, tasks_seed)
        else:
            hist, record = train_standard_method(method, backbone_name,  

            ↪seed, tasks_seed)

        all_histories.append(hist)
        run_records.append(record)

history_df = pd.concat(all_histories, ignore_index=True)
runs_df = pd.DataFrame(run_records)

print("\nDONE")
display(history_df.head())
display(runs_df.head())

```

Run 1: seed=0, backbone=mlp, method=finetune

Run 2: seed=0, backbone=mlp, method=joint_upper_bound

Run 3: seed=0, backbone=mlp, method=ewc

Run 4: seed=0, backbone=mlp, method=lfw

Run 5: seed=0, backbone=mlp, method=experience_replay

Run 6: seed=0, backbone=mlp, method=pnn

Run 7: seed=0, backbone=mlp, method=sparse_moe

Run 8: seed=0, backbone=mlp, method=mmals_full_functional_memory

DONE

	seed	method	backbone	after_task	task_id	task_name	acc
0	0	finetune	mlp	0	0	0 vs 1	0.970
1	0	finetune	mlp	1	0	0 vs 1	0.965
2	0	finetune	mlp	1	1	2 vs 3	0.940
3	0	finetune	mlp	2	0	0 vs 1	0.940
4	0	finetune	mlp	2	1	2 vs 3	0.915

	seed	method	backbone	param_count	param_mb_fp32	train_time_s	\
0	0	finetune	mlp	218058	0.831825	0.411621	
1	0	joint_upper_bound	mlp	218058	0.831825	1.017846	
2	0	ewc	mlp	218058	0.831825	0.460453	
3	0	lwf	mlp	218058	0.831825	0.651743	
4	0	experience_replay	mlp	218058	0.831825	0.491117	

	peak_gpu_mb	replay_buffer_items
0	NaN	0
1	NaN	0
2	NaN	0
3	NaN	0
4	NaN	1000

11 10. Scorecard

The scorecard is the core output of v0.11.

A method is not better because it has one good accuracy number.

It is stronger only if it improves stability at acceptable cost.

```
[11]: # =====
# 10. Scorecard
# =====

group_cols = ["seed", "method", "backbone"]

final_eval_df, per_seed_acc_df, task_forgetting_df, per_seed_forgetting_df = \
    compute_final_metrics(
        history_df=history_df,
        group_cols=group_cols,
    )

acc_summary = (
    per_seed_acc_df
    .groupby(["method", "backbone"])["final_mean_acc"]
    .agg(["mean", "std"])
    .reset_index()
    .rename(columns={"mean": "final_acc_mean", "std": "final_acc_std"})
)

forgetting_summary = (
    per_seed_forgetting_df
    .groupby(["method", "backbone"])["avg_forgetting"]
    .agg(["mean", "std"])
    .reset_index()
    .rename(columns={"mean": "forgetting_mean", "std": "forgetting_std"})
)
```

```

cost_summary = (
    runs_df
    .groupby(["method", "backbone"])
    .agg(
        param_count_mean=("param_count", "mean"),
        param_mb_mean=("param_mb_fp32", "mean"),
        train_time_s_mean=("train_time_s", "mean"),
        peak_gpu_mb_mean=("peak_gpu_mb", "mean"),
        replay_buffer_items_mean=("replay_buffer_items", "mean"),
    )
    .reset_index()
)

scorecard = (
    acc_summary
    .merge(forgetting_summary, on=["method", "backbone"], how="left")
    .merge(cost_summary, on=["method", "backbone"], how="left")
)

scorecard["stability_score"] = scorecard["final_acc_mean"] -
    ↪scorecard["forgetting_mean"]
scorecard["efficiency_score"] = scorecard["stability_score"] / np.
    ↪log1p(scorecard["train_time_s_mean"].fillna(1.0))

scorecard = scorecard.sort_values(
    ["stability_score", "efficiency_score"],
    ascending=[False, False]
)

display(scorecard)
display(task_forgetting_df.head())

```

	method	backbone	final_acc_mean	final_acc_std	\
3	joint_upper_bound	mlp	0.980	NaN	
6	pnn	mlp	0.979	NaN	
1	experience_replay	mlp	0.971	NaN	
5	mmals_full_functional_memory	mmals	0.967	NaN	
4	lwf	mlp	0.962	NaN	
0	ewc	mlp	0.873	NaN	
2	finetune	mlp	0.821	NaN	
7	sparse_moe	mlp	0.795	NaN	

	forgetting_mean	forgetting_std	param_count_mean	param_mb_mean	\
3	0.000	NaN	218058.0	0.831825	
6	0.000	NaN	217538.0	0.829842	
1	0.002	NaN	218058.0	0.831825	

5	0.003	NaN	1232543.0	4.701778
4	0.003	NaN	218058.0	0.831825
0	0.102	NaN	218058.0	0.831825
2	0.142	NaN	218058.0	0.831825
7	0.176	NaN	1190943.0	4.543087

	train_time_s_mean	peak_gpu_mb_mean	replay_buffer_items_mean	\
3	1.017846	NaN	0.0	
6	0.373760	NaN	0.0	
1	0.491117	NaN	1000.0	
5	3.991640	NaN	0.0	
4	0.651743	NaN	0.0	
0	0.460453	NaN	0.0	
2	0.411621	NaN	0.0	
7	1.719743	NaN	0.0	

	stability_score	efficiency_score
3	0.980	1.395951
6	0.979	3.082964
1	0.969	2.425377
5	0.964	0.599590
4	0.959	1.911000
0	0.771	2.035660
2	0.679	1.969609
7	0.619	0.618668

	seed	method	backbone	task_id	task_name	best_acc	final_acc	\
0	0	ewc	mlp	0	0 vs 1	0.970	0.905	
1	0	ewc	mlp	1	2 vs 3	0.940	0.500	
2	0	ewc	mlp	2	4 vs 5	0.990	0.985	
3	0	ewc	mlp	3	6 vs 7	1.000	1.000	
4	0	ewc	mlp	4	8 vs 9	0.975	0.975	

	per_task_forgetting
0	0.065
1	0.440
2	0.005
3	0.000
4	0.000

12 11. Interpretation rules

12.1 If MMALS beats fine-tuning but not replay/EWC/LwF

Then MMALS is promising internally, but not yet externally competitive.

12.2 If MMALS beats EWC/SI/LwF but not replay

Then MMALS is a strong replay-free memory candidate.

12.3 If MMALS beats replay at similar or lower memory

Then MMALS becomes a serious continual-learning candidate.

12.4 If MoE matches MMALS

Then the claim must shift from “mycelial memory” to “modular routing helps.”

12.5 If PNN matches MMALS but uses many more parameters

Then MMALS may be valuable as a compact structural-memory alternative.

12.6 If CNN/ResNet/ViT backbones dominate MMALS

Then MMALS must improve its host architecture before claiming production relevance.

12.7 If MMALS has lower forgetting but much higher cost

Then MMALS is a risk-reduction mechanism, useful mainly for high-stakes systems.

13 12. Optional graph-learning track

Graph benchmarks are not directly comparable to Fashion-MNIST/CIFAR.

Use this section only if MMALS is adapted to graph-structured data.

The correct comparison is:

- GCN / GraphSAGE / GAT on Cora, Citeseer, PubMed
- graph continual learning scenarios
- MMALS with graph hosts or fungal GNN medium

```
[12]: # =====  
# 12. Optional GNN benchmark scaffold  
# =====  
  
RUN_GRAPH_TRACK = False  
  
if RUN_GRAPH_TRACK:  
    try:  
        import torch_geometric  
        from torch_geometric.datasets import Planetoid  
        from torch_geometric.nn import GCNConv  
  
        class SimpleGCN(nn.Module):  
            def __init__(self, in_dim, hidden_dim, out_dim):  
                super().__init__()  
                self.conv1 = GCNConv(in_dim, hidden_dim)  
                self.conv2 = GCNConv(hidden_dim, out_dim)  
  
            def forward(self, x, edge_index):
```



```

        x = F.relu(self.conv1(x, edge_index))
        x = self.conv2(x, edge_index)
        return x

dataset = Planetoid(root="./graph_data", name="Cora")
data = dataset[0].to(DEVICE)
model = SimpleGCN(dataset.num_node_features, 64, dataset.num_classes).
→to(DEVICE)
opt = torch.optim.Adam(model.parameters(), lr=0.01, weight_decay=5e-4)

for epoch in range(200):
    model.train()
    opt.zero_grad()
    out = model(data.x, data.edge_index)
    loss = F.cross_entropy(out[data.train_mask], data.y[data.
→train_mask])
    loss.backward()
    opt.step()

    model.eval()
    pred = model(data.x, data.edge_index).argmax(dim=1)
    acc = (pred[data.test_mask] == data.y[data.test_mask]).float().mean().
→item()
    print("Cora GCN test accuracy:", acc)

except Exception as e:
    print("Graph track unavailable. Install torch_geometric for this_
→section.")
    print("Error:", e)

```

14 13. Export results

This produces a folder and ZIP containing:

- history_df.csv
- runs_df.csv
- scorecard.csv
- task_forgetting_df.csv
- final_eval_df.csv

```

[13]: # =====
# 13. Export results
# =====

RESULTS_DIR = Path("./mmals_v011_comparative_results")
RESULTS_DIR.mkdir(exist_ok=True)

```

```

history_df.to_csv(RESULTS_DIR / "history_df.csv", index=False)
runs_df.to_csv(RESULTS_DIR / "runs_df.csv", index=False)
scorecard.to_csv(RESULTS_DIR / "scorecard.csv", index=False)
task_forgetting_df.to_csv(RESULTS_DIR / "task_forgetting_df.csv", index=False)
final_eval_df.to_csv(RESULTS_DIR / "final_eval_df.csv", index=False)

import shutil
zip_path = shutil.make_archive(RESULTS_DIR.name, "zip", RESULTS_DIR)

print("Saved results to:", RESULTS_DIR.resolve())
print("Compressed results to:", zip_path)

```

Saved results to: /content/mmals_v011_comparative_results

Compressed results to: /content/mmals_v011_comparative_results.zip

15 14. Conservative paper statement template

After running v0.11, use a calibrated statement like this:

MMALS v0.11 evaluates whether the functional mycelial memory mechanism remains competitive when compared against standard continual-learning baselines, including regularization, replay, expansion, modular routing, and standard neural backbones. We report not only final accuracy but also forgetting, per-task stability, parameter growth, train time, inference latency, GPU memory, and memory-buffer overhead. The objective is to determine whether the MMALS mechanism provides an advantage beyond modular capacity, replay, distillation, or routing alone.

Do not claim that MMALS solves continual learning unless it beats strong baselines across harder benchmarks and multiple seeds.

16 Resources

This is a placeholder for any additional resources or references.

```
[14]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[ ]: import os

!apt-get update
!apt-get install texlive texlive-xetex texlive-latex-extra pandoc
!pip install py pandoc
!pip -q install nbformat
```

Get:1 <http://security.ubuntu.com/ubuntu> jammy-security InRelease [129 kB]

Hit:2 <http://archive.ubuntu.com/ubuntu> jammy InRelease

```

Get:3 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:5 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease
[3,632 B]
Get:6 https://cli.github.com/packages stable InRelease [3,917 B]
Get:7 https://r2u.stat.illinois.edu/ubuntu jammy InRelease [6,555 B]
Get:8 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
[18.1 kB]
Get:9 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages
[1,294 kB]
Hit:10 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Get:11 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64
Packages [7,004 kB]
Get:12 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages
[3,915 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages
[1,602 kB]
Get:14 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [4,247
kB]
Get:15 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages
[7,251 kB]
Get:16 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ Packages [93.4
kB]
Get:17 https://r2u.stat.illinois.edu/ubuntu jammy/main amd64 Packages [3,003 kB]
Get:18 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy/main amd64
Packages [38.9 kB]
Get:19 https://r2u.stat.illinois.edu/ubuntu jammy/main all Packages [10.2 MB]
Fetched 39.0 MB in 4s (9,650 kB/s)
Reading package lists... Done
W: Skipping acquire of configured file 'main/source/Sources' as repository
'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide
it (sources.list entry misspelt?)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
pandoc is already the newest version (2.9.2.1-3ubuntu2).
pandoc set to manually installed.
The following additional packages will be installed:
  dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
  fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java
  libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12
  libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0
  libsyntax2 libteckit0 libtexlua53 libtexluajit2 libwoff1 libzzip-0-13
  lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
  ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration t1utils
  teckit tex-common tex-gyre texlive-base texlive-binaries
  texlive-fonts-recommended texlive-latex-base texlive-latex-recommended
  texlive-pictures texlive-plain-generic tipa xfonts-encodings xfonts-utils

```

Suggested packages:

```
fonts-noto fonts-freefont-otf | fonts-freefont-ttf libavalon-framework-java
libcommons-logging-java-doc libexcalibur-logkit-java liblog4j1.2-java
poppler-utils ghostscript fonts-japanese-mincho | fonts-ipafont-mincho
fonts-japanese-gothic | fonts-ipafont-gothic fonts-arphic-ukai
fonts-arphic-uming fonts-nanum ri ruby-dev bundler debhelper gv
| postscript-viewer perl-tk xpdf | pdf-viewer xzdec
texlive-fonts-recommended-doc texlive-latex-base-doc python3-pygments
icc-profiles libfile-which-perl libspreadsheet-parseexcel-perl
texlive-latex-extra-doc texlive-latex-recommended-doc texlive-luatex
texlive-pstricks dot2tex prerex texlive-pictures-doc vprerex
default-jre-headless tipa-doc
```

The following NEW packages will be installed:

```
dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java
libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12
libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0
libsyntax2 libteckit0 libtexlua53 libtexluajit2 libwoff1 libzip-0-13
lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration t1utils
teckit tex-common tex-gyre texlive texlive-base texlive-binaries
texlive-fonts-recommended texlive-latex-base texlive-latex-extra
texlive-latex-recommended texlive-pictures texlive-plain-generic
texlive-xetex tipa xfonts-encodings xfonts-utils
```

0 upgraded, 54 newly installed, 0 to remove and 17 not upgraded.

Need to get 182 MB of archives.

After this operation, 571 MB of additional disk space will be used.

Get:1 <http://archive.ubuntu.com/ubuntu/jammy/main amd64 fonts-droid-fallback all 1:6.0.1r16-1.1build1> [1,805 kB]

Get:2 <http://archive.ubuntu.com/ubuntu/jammy/main amd64 fonts-lato all 2.0-2.1> [2,696 kB]

Get:3 <http://archive.ubuntu.com/ubuntu/jammy/main amd64 poppler-data all 0.4.11-1> [2,171 kB]

Get:4 <http://archive.ubuntu.com/ubuntu/jammy/universe amd64 tex-common all 6.17> [33.7 kB]

Get:5 <http://archive.ubuntu.com/ubuntu/jammy/main amd64 fonts-urw-base35 all 20200910-1> [6,367 kB]

Get:6 <http://archive.ubuntu.com/ubuntu/jammy-updates/main amd64 libgs9-common all 9.55.0~dfsg1-0ubuntu5.13> [753 kB]

Get:7 <http://archive.ubuntu.com/ubuntu/jammy-updates/main amd64 libidn12 amd64 1.38-4ubuntu1> [60.0 kB]

Get:8 <http://archive.ubuntu.com/ubuntu/jammy/main amd64 libijs-0.35 amd64 0.35-15build2> [16.5 kB]

Get:9 <http://archive.ubuntu.com/ubuntu/jammy/main amd64 libjbig2dec0 amd64 0.19-3build2> [64.7 kB]

Get:10 <http://archive.ubuntu.com/ubuntu/jammy-updates/main amd64 libgs9 amd64 9.55.0~dfsg1-0ubuntu5.13> [5,032 kB]

Get:11 <http://archive.ubuntu.com/ubuntu/jammy-updates/main amd64 libkpathsea6>

amd64 2021.20210626.59705-1ubuntu0.3 [60.6 kB]
 Get:12 http://archive.ubuntu.com/ubuntu jammy/main amd64 libwoff1 amd64
 1.0.2-1build4 [45.2 kB]
 Get:13 http://archive.ubuntu.com/ubuntu jammy/universe amd64 dvisvgm amd64
 2.13.1-1 [1,221 kB]
 Get:14 http://archive.ubuntu.com/ubuntu jammy/universe amd64 fonts-lmodern all
 2.004.5-6.1 [4,532 kB]
 Get:15 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-noto-mono all
 20201225-1build1 [397 kB]
 Get:16 http://archive.ubuntu.com/ubuntu jammy/universe amd64 fonts-texgyre all
 20180621-3.1 [10.2 MB]
 Get:17 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libapache-pom-java
 all 18-1 [4,720 B]
 Get:18 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libcommons-parent-
 java all 43-1 [10.8 kB]
 Get:19 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libcommons-logging-
 java all 1.2-2 [60.3 kB]
 Get:20 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libptexenc1
 amd64 2021.20210626.59705-1ubuntu0.3 [39.1 kB]
 Get:21 http://archive.ubuntu.com/ubuntu jammy/main amd64 rubygems-integration
 all 1.18 [5,336 B]
 Get:22 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 ruby3.0 amd64
 3.0.2-7ubuntu2.12 [50.1 kB]
 Get:23 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 ruby-rubygems
 all 3.3.5-2ubuntu1.2 [228 kB]
 Get:24 http://archive.ubuntu.com/ubuntu jammy/main amd64 ruby amd64 1:3.0~exp1
 [5,100 B]
 Get:25 http://archive.ubuntu.com/ubuntu jammy/main amd64 rake all 13.0.6-2 [61.7
 kB]
 Get:26 http://archive.ubuntu.com/ubuntu jammy/main amd64 ruby-net-telnet all
 0.1.1-2 [12.6 kB]
 Get:27 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 ruby-webrick
 all 1.7.0-3ubuntu0.2 [52.5 kB]
 Get:28 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 ruby-xmlrpc all
 0.3.2-1ubuntu0.1 [24.9 kB]
 Get:29 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libruby3.0
 amd64 3.0.2-7ubuntu2.12 [5,113 kB]
 Get:30 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libsynchronet2
 amd64 2021.20210626.59705-1ubuntu0.3 [55.8 kB]
 Get:31 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libteckit0 amd64
 2.5.11+ds1-1 [421 kB]
 Get:32 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libtexlua53
 amd64 2021.20210626.59705-1ubuntu0.3 [120 kB]
 Get:33 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libtexluajit2
 amd64 2021.20210626.59705-1ubuntu0.3 [267 kB]
 Get:34 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libzip-0-13 amd64
 0.13.72+dfsg.1-1.1 [27.0 kB]
 Get:35 http://archive.ubuntu.com/ubuntu jammy/main amd64 xfonts-encodings all

```

1:1.0.5-0ubuntu2 [578 kB]
Get:36 http://archive.ubuntu.com/ubuntu jammy/main amd64 xfonts-utils amd64
1:7.7+6build2 [94.6 kB]
Get:37 http://archive.ubuntu.com/ubuntu jammy/universe amd64 lmodern all
2.004.5-6.1 [9,471 kB]
Get:38 http://archive.ubuntu.com/ubuntu jammy/universe amd64 preview-latex-style
all 12.2-1ubuntu1 [185 kB]
Get:39 http://archive.ubuntu.com/ubuntu jammy/main amd64 t1utils amd64
1.41-4build2 [61.3 kB]
Get:40 http://archive.ubuntu.com/ubuntu jammy/universe amd64 teckit amd64
2.5.11+ds1-1 [699 kB]
Get:41 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tex-gyre all
20180621-3.1 [6,209 kB]
Get:42 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 texlive-
binaries amd64 2021.20210626.59705-1ubuntu0.3 [9,861 kB]
Get:43 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-base all
2021.20220204-1 [21.0 MB]
Get:44 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-fonts-
recommended all 2021.20220204-1 [4,972 kB]
Get:45 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-base
all 2021.20220204-1 [1,128 kB]
Get:46 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-
recommended all 2021.20220204-1 [14.4 MB]
Get:47 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive all
2021.20220204-1 [14.3 kB]
Get:48 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libfontbox-java all
1:1.8.16-2 [207 kB]
Get:49 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libpdfbox-java all
1:1.8.16-2 [5,199 kB]
Get:50 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-pictures
all 2021.20220204-1 [8,720 kB]
Get:51 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-extra
all 2021.20220204-1 [13.9 MB]
Get:52 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-plain-
generic all 2021.20220204-1 [27.5 MB]
Get:53 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tipa all 2:1.3-21
[2,967 kB]
Get:54 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-xetex all
2021.20220204-1 [12.4 MB]
Fetched 182 MB in 3s (52.7 MB/s)
Extracting templates from packages: 100%
Preconfiguring packages ...
Selecting previously unselected package fonts-droid-fallback.
(Reading database ... 118251 files and directories currently installed.)
Preparing to unpack .../00-fonts-droid-fallback_1%3a6.0.1r16-1.1build1_all.deb
...
Unpacking fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Selecting previously unselected package fonts-lato.

```

```

Preparing to unpack .../01-fonts-lato_2.0-2.1_all.deb ...
Unpacking fonts-lato (2.0-2.1) ...
Selecting previously unselected package poppler-data.
Preparing to unpack .../02-poppler-data_0.4.11-1_all.deb ...
Unpacking poppler-data (0.4.11-1) ...
Selecting previously unselected package tex-common.
Preparing to unpack .../03-tex-common_6.17_all.deb ...
Unpacking tex-common (6.17) ...
Selecting previously unselected package fonts-urw-base35.
Preparing to unpack .../04-fonts-urw-base35_20200910-1_all.deb ...
Unpacking fonts-urw-base35 (20200910-1) ...
Selecting previously unselected package libgs9-common.
Preparing to unpack .../05-libgs9-common_9.55.0~dfsg1-0ubuntu5.13_all.deb ...
Unpacking libgs9-common (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libidn12:amd64.
Preparing to unpack .../06-libidn12_1.38-4ubuntu1_amd64.deb ...
Unpacking libidn12:amd64 (1.38-4ubuntu1) ...
Selecting previously unselected package libijs-0.35:amd64.
Preparing to unpack .../07-libijs-0.35_0.35-15build2_amd64.deb ...
Unpacking libijs-0.35:amd64 (0.35-15build2) ...
Selecting previously unselected package libjbig2dec0:amd64.
Preparing to unpack .../08-libjbig2dec0_0.19-3build2_amd64.deb ...
Unpacking libjbig2dec0:amd64 (0.19-3build2) ...
Selecting previously unselected package libgs9:amd64.
Preparing to unpack .../09-libgs9_9.55.0~dfsg1-0ubuntu5.13_amd64.deb ...
Unpacking libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libkpathsea6:amd64.
Preparing to unpack .../10-libkpathsea6_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libwoff1:amd64.
Preparing to unpack .../11-libwoff1_1.0.2-1build4_amd64.deb ...
Unpacking libwoff1:amd64 (1.0.2-1build4) ...
Selecting previously unselected package dvisvgm.
Preparing to unpack .../12-dvisvgm_2.13.1-1_amd64.deb ...
Unpacking dvisvgm (2.13.1-1) ...
Selecting previously unselected package fonts-lmodern.
Preparing to unpack .../13-fonts-lmodern_2.004.5-6.1_all.deb ...
Unpacking fonts-lmodern (2.004.5-6.1) ...
Selecting previously unselected package fonts-noto-mono.
Preparing to unpack .../14-fonts-noto-mono_20201225-1build1_all.deb ...
Unpacking fonts-noto-mono (20201225-1build1) ...
Selecting previously unselected package fonts-texgyre.
Preparing to unpack .../15-fonts-texgyre_20180621-3.1_all.deb ...
Unpacking fonts-texgyre (20180621-3.1) ...
Selecting previously unselected package libapache-pom-java.
Preparing to unpack .../16-libapache-pom-java_18-1_all.deb ...
Unpacking libapache-pom-java (18-1) ...

```

```

Selecting previously unselected package libcommons-parent-java.
Preparing to unpack .../17-libcommons-parent-java_43-1_all.deb ...
Unpacking libcommons-parent-java (43-1) ...
Selecting previously unselected package libcommons-logging-java.
Preparing to unpack .../18-libcommons-logging-java_1.2-2_all.deb ...
Unpacking libcommons-logging-java (1.2-2) ...
Selecting previously unselected package libptexenc1:amd64.
Preparing to unpack .../19-libptexenc1_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package rubygems-integration.
Preparing to unpack .../20-rubygems-integration_1.18_all.deb ...
Unpacking rubygems-integration (1.18) ...
Selecting previously unselected package ruby3.0.
Preparing to unpack .../21-ruby3.0_3.0.2-7ubuntu2.12_amd64.deb ...
Unpacking ruby3.0 (3.0.2-7ubuntu2.12) ...
Selecting previously unselected package ruby-rubygems.
Preparing to unpack .../22-ruby-rubygems_3.3.5-2ubuntu1.2_all.deb ...
Unpacking ruby-rubygems (3.3.5-2ubuntu1.2) ...
Selecting previously unselected package ruby.
Preparing to unpack .../23-ruby_1%3a3.0~exp1_amd64.deb ...
Unpacking ruby (1:3.0~exp1) ...
Selecting previously unselected package rake.
Preparing to unpack .../24-rake_13.0.6-2_all.deb ...
Unpacking rake (13.0.6-2) ...
Selecting previously unselected package ruby-net-telnet.
Preparing to unpack .../25-ruby-net-telnet_0.1.1-2_all.deb ...
Unpacking ruby-net-telnet (0.1.1-2) ...
Selecting previously unselected package ruby-webrick.
Preparing to unpack .../26-ruby-webrick_1.7.0-3ubuntu0.2_all.deb ...
Unpacking ruby-webrick (1.7.0-3ubuntu0.2) ...
Selecting previously unselected package ruby-xmlrpc.
Preparing to unpack .../27-ruby-xmlrpc_0.3.2-1ubuntu0.1_all.deb ...
Unpacking ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Selecting previously unselected package libruby3.0:amd64.
Preparing to unpack .../28-libruby3.0_3.0.2-7ubuntu2.12_amd64.deb ...
Unpacking libruby3.0:amd64 (3.0.2-7ubuntu2.12) ...
Selecting previously unselected package libsyntax2:amd64.
Preparing to unpack .../29-libsyntax2_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libsyntax2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libteckit0:amd64.
Preparing to unpack .../30-libteckit0_2.5.11+ds1-1_amd64.deb ...
Unpacking libteckit0:amd64 (2.5.11+ds1-1) ...
Selecting previously unselected package libtexlua53:amd64.
Preparing to unpack .../31-libtexlua53_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.3) ...

```



```

Selecting previously unselected package libtexluaajit2:amd64.
Preparing to unpack
.../32-libtexluaajit2_2021.20210626.59705-1ubuntu0.3_amd64.deb ...
Unpacking libtexluaajit2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libzzip-0-13:amd64.
Preparing to unpack .../33-libzzip-0-13_0.13.72+dfsg.1-1.1_amd64.deb ...
Unpacking libzzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Selecting previously unselected package xfonts-encodings.
Preparing to unpack .../34-xfonts-encodings_1%3a1.0.5-0ubuntu2_all.deb ...
Unpacking xfonts-encodings (1:1.0.5-0ubuntu2) ...
Selecting previously unselected package xfonts-utils.
Preparing to unpack .../35-xfonts-utils_1%3a7.7+6build2_amd64.deb ...
Unpacking xfonts-utils (1:7.7+6build2) ...
Selecting previously unselected package lmodern.
Preparing to unpack .../36-lmodern_2.004.5-6.1_all.deb ...
Unpacking lmodern (2.004.5-6.1) ...
Selecting previously unselected package preview-latex-style.
Preparing to unpack .../37-preview-latex-style_12.2-1ubuntu1_all.deb ...
Unpacking preview-latex-style (12.2-1ubuntu1) ...
Selecting previously unselected package tlutils.
Preparing to unpack .../38-tlutils_1.41-4build2_amd64.deb ...
Unpacking tlutils (1.41-4build2) ...
Selecting previously unselected package teckit.
Preparing to unpack .../39-teckit_2.5.11+ds1-1_amd64.deb ...
Unpacking teckit (2.5.11+ds1-1) ...
Selecting previously unselected package tex-gyre.
Preparing to unpack .../40-tex-gyre_20180621-3.1_all.deb ...
Unpacking tex-gyre (20180621-3.1) ...
Selecting previously unselected package texlive-binaries.
Preparing to unpack .../41-texlive-
binaries_2021.20210626.59705-1ubuntu0.3_amd64.deb ...
Unpacking texlive-binaries (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package texlive-base.
Preparing to unpack .../42-texlive-base_2021.20220204-1_all.deb ...
Unpacking texlive-base (2021.20220204-1) ...
Selecting previously unselected package texlive-fonts-recommended.
Preparing to unpack .../43-texlive-fonts-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-fonts-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-base.
Preparing to unpack .../44-texlive-latex-base_2021.20220204-1_all.deb ...
Unpacking texlive-latex-base (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-recommended.
Preparing to unpack .../45-texlive-latex-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-latex-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive.
Preparing to unpack .../46-texlive_2021.20220204-1_all.deb ...
Unpacking texlive (2021.20220204-1) ...
Selecting previously unselected package libfontbox-java.

```

```

Preparing to unpack .../47-libfontbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libfontbox-java (1:1.8.16-2) ...
Selecting previously unselected package libpdfbox-java.
Preparing to unpack .../48-libpdfbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libpdfbox-java (1:1.8.16-2) ...
Selecting previously unselected package texlive-pictures.
Preparing to unpack .../49-texlive-pictures_2021.20220204-1_all.deb ...
Unpacking texlive-pictures (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-extra.
Preparing to unpack .../50-texlive-latex-extra_2021.20220204-1_all.deb ...
Unpacking texlive-latex-extra (2021.20220204-1) ...
Selecting previously unselected package texlive-plain-generic.
Preparing to unpack .../51-texlive-plain-generic_2021.20220204-1_all.deb ...
Unpacking texlive-plain-generic (2021.20220204-1) ...
Selecting previously unselected package tipa.
Preparing to unpack .../52-tipa_2%3a1.3-21_all.deb ...
Unpacking tipa (2:1.3-21) ...
Selecting previously unselected package texlive-xetex.
Preparing to unpack .../53-texlive-xetex_2021.20220204-1_all.deb ...
Unpacking texlive-xetex (2021.20220204-1) ...
Setting up fonts-lato (2.0-2.1) ...
Setting up fonts-noto-mono (20201225-1build1) ...
Setting up libwoff1:amd64 (1.0.2-1build4) ...
Setting up libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up libijs-0.35:amd64 (0.35-15build2) ...
Setting up libtexluajit2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up libfontbox-java (1:1.8.16-2) ...
Setting up rubygems-integration (1.18) ...
Setting up libzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Setting up fonts-urw-base35 (20200910-1) ...
Setting up poppler-data (0.4.11-1) ...
Setting up tex-common (6.17) ...
update-language: texlive-base not installed and configured, doing nothing!
Setting up libjbig2dec0:amd64 (0.19-3build2) ...
Setting up libteckit0:amd64 (2.5.11+ds1-1) ...
Setting up libapache-pom-java (18-1) ...
Setting up ruby-net-telnet (0.1.1-2) ...
Setting up xfonts-encodings (1:1.0.5-0ubuntu2) ...
Setting up t1utils (1.41-4build2) ...
Setting up libidn12:amd64 (1.38-4ubuntu1) ...
Setting up fonts-texgyre (20180621-3.1) ...
Setting up libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up ruby-webrick (1.7.0-3ubuntu0.2) ...
Setting up fonts-lmodern (2.004.5-6.1) ...
Setting up fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Setting up ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Setting up libsynchronet2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up libgs9-common (9.55.0~dfsg1-0ubuntu5.13) ...

```

```

Setting up teckit (2.5.11+ds1-1) ...
Setting up libpdfbox-java (1:1.8.16-2) ...
Setting up libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) ...
Setting up preview-latex-style (12.2-1ubuntu1) ...
Setting up libcommons-parent-java (43-1) ...
Setting up dvisvgm (2.13.1-1) ...
Setting up libcommons-logging-java (1.2-2) ...
Setting up xfonts-utils (1:7.7+6build2) ...
Setting up libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up texlive-binaries (2021.20210626.59705-1ubuntu0.3) ...
update-alternatives: using /usr/bin/xdvi-xaw to provide /usr/bin/xdvi.bin
(xdvi.bin) in auto mode
update-alternatives: using /usr/bin/bibtex.original to provide /usr/bin/bibtex
(bibtex) in auto mode
Setting up lmodern (2.004.5-6.1) ...
Setting up texlive-base (2021.20220204-1) ...
/usr/bin/ucfr
/usr/bin/ucfr
/usr/bin/ucfr
/usr/bin/ucfr
mktexlsr: Updating /var/lib/texmf/ls-R-TEXLIVEDIST...
mktexlsr: Updating /var/lib/texmf/ls-R-TEXMFMAIN...
mktexlsr: Updating /var/lib/texmf/ls-R...
mktexlsr: Done.
tl-paper: setting paper size for dvips to a4:
/var/lib/texmf/dvips/config/config-paper.ps
tl-paper: setting paper size for dvipdfmx to a4:
/var/lib/texmf/dvipdfmx/dvipdfmx-paper.cfg
tl-paper: setting paper size for xdvi to a4: /var/lib/texmf/xdvi/XDvi-paper
tl-paper: setting paper size for pdftex to a4: /var/lib/texmf/tex/generic/tex-
ini-files/pdftexconfig.tex
Setting up tex-gyre (20180621-3.1) ...
Setting up texlive-plain-generic (2021.20220204-1) ...
Setting up texlive-latex-base (2021.20220204-1) ...
Setting up texlive-latex-recommended (2021.20220204-1) ...
Setting up texlive-pictures (2021.20220204-1) ...
Setting up texlive-fonts-recommended (2021.20220204-1) ...
Setting up tipa (2:1.3-21) ...
Setting up texlive (2021.20220204-1) ...
Setting up texlive-latex-extra (2021.20220204-1) ...
Setting up texlive-xetex (2021.20220204-1) ...
Setting up rake (13.0.6-2) ...
Setting up libruby3.0:amd64 (3.0.2-7ubuntu2.12) ...
Setting up ruby3.0 (3.0.2-7ubuntu2.12) ...
Setting up ruby (1:3.0~exp1) ...
Setting up ruby-rubygems (3.3.5-2ubuntu1.2) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for mailcap (3.70+nm1ubuntu1) ...

```

```

Processing triggers for fontconfig (2.13.1-4.2ubuntu5) ...
Processing triggers for libc-bin (2.35-0ubuntu3.13) ...
/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_5.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libhwloc.so.15 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc_proxy.so.2 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc.so.2 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbb.so.12 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm_debug.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_loader.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_opencl.so.0 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_0.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero_v2.so.0 is not a
symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero.so.0 is not a
symbolic link

/sbin/ldconfig.real: /usr/local/lib/libumf.so.1 is not a symbolic link

Processing triggers for tex-common (6.17) ...
Running updmap-sys. This may take some time... done.
Running mktexlsr /var/lib/texmf ... done.
Building format(s) --all.
    This may take some time...

```

```

[ ]: # Explicitly setting the notebook filename for this execution,
     # as there isn't a robust automatic detection method in Colab.
notebook_filename = "MMALS_Prototype_0_11_Comparative_Evidence_Harness.ipynb"
notebook_name_without_ext = os.path.splitext(notebook_filename)[0]

# Define the full path to the notebook in Google Drive

```

```
full_notebook_path_drive = f"/content/drive/MyDrive/Colab Notebooks/  
↳{notebook_filename}"  
local_notebook_path = f"/content/{notebook_filename}"  
  
# Copy the notebook to the local /content/ directory to avoid path issues with  
↳nbconvert  
print(f"Executing: cp \"{full_notebook_path_drive}\" \"{local_notebook_path}\"")  
!bash -c "cp \"{full_notebook_path_drive}\" \"{local_notebook_path}\""
```